LAB3 SOLUTIONS

SIFT & Local Visual Features Matching

Ex. 1.a - Ratio Test

```
# Detect and compute keypoints and descriptors
# keypoints, descriptors = cv2.Feature2D.detectAndCompute( image, mask[,
    descriptors[, useProvidedKeypoints]] )
query_keypoints, query_descriptors = sift.detectAndCompute(query_image, None)
db_keypoints, db_descriptors = sift.detectAndCompute(db_image, None)
print(f"Query image descriptors shape: {query_descriptors.shape}, database image
    descriptors shape: {db_descriptors.shape}")

# Pairwise distances between query and database descriptors
dist_matrix = sklearn.metrics.pairwise_distances(query_descriptors, db_descriptors)
print(f"Distance matrix shape: {dist_matrix.shape}")

## K-NN matcher (K=2)
# For each query img descriptor find the distances and indices (db_idx) of the 2
    closest database img descriptors.
k = 2

distances = np.partition(dist_matrix, kth=(k-1), axis=1)[:, :k]
db_idx = np.argpartition(dist_matrix, kth=(k-1), axis=1)[:, :k]
print(f"Distance matrix of each query image descriptor and its 2 closest database
    image descriptors: {distances.shape}")

# Implement ratio test using distances and db_idx
min_dist = [] # dist(i,j) that pass the test
min_query_idx = [] # indices of the query descriptors that pass the test
min_db_idx = [] # indices of the database descriptors that pass the test

for i in range(distances.shape[0]):
  if distances[i][0] < 0.7 * distances[i][1]:
    min_dist.append(distances[i][0])
    min_query_idx.append(i)
    min_db_idx.append(db_idx[i][0])
```

Ex. 1.b - Change the constant parameter r (0.70) in the ratio test.**

What happens if $r \to 1$*? What happens if* $r \to 0$*?*

When $\frac{dist(i,j)}{dist(i,k)} < 1 \to$, $dist(i,j) < dist(i,k)$ which is always true since des_{db_k} is the *second* closest database descriptor to query descriptor des_{q_i}, so all matches are accepted.
When $\frac{dist(i,j)}{dist(i,k)} < a$ with $a \to 0$, then all matches are discarded.

Ex. 2 - Brute-force Matcher with threshold distance

```
# Get keypoints and descriptors
# SIFT returns a 128-dimensional feature vector for each key point
query_keypoints, query_descriptors = sift.detectAndCompute(query_image, None)
db_keypoints, db_descriptors = sift.detectAndCompute(database_image, None)
```

```
# Brute-force matcher
bf = cv2.BFMatcher_create(cv2.NORM_L2)

# Match descriptors
matches = bf.match(query_descriptors, db_descriptors)

# Discard matches using the following distance thresholds:
# 1. low threshold: distance below 100
thrs1 = 100
matched_des = [matches[idx] for idx in range(len(matches)) if matches[idx].distance
    <= thrs1]
# 2. intermediate threshold: distance below 300
thrs2 = 300
matched_des_int = [matches[idx] for idx in range(len(matches)) if matches[idx].
    distance <= thrs2]
# 3. outliers threshold: distance above 200
matched_des_out = [matches[idx] for idx in range(len(matches)) if matches[idx].
    distance > 200]
```